

Introducción a R y RStudio

R y Bases de Datos

Índice

Ideas clave	3
8.1. Introducción y Objetivos	3
8.2. Conexión de R con diferentes bases de datos	5
8.3. Conexión a MySQL	7
8.4. Conexión a MongoDB	10
8.5. Manipulación y Análisis de Datos	16
8.6. Visualización de Datos y Reporting	20
8.7. Referencias bibliográficas	26
8.8. Cuaderno de ejercicios	27

8.1. Introducción y Objetivos

En este tema, nos adentraremos en una de las aplicaciones más potentes y útiles de R: la conexión con bases de datos. En el contexto de la ciencia de datos, la capacidad de acceder y manipular grandes volúmenes de datos almacenados en bases de datos es fundamental. Las bases de datos permiten almacenar, organizar y gestionar grandes cantidades de información de una manera eficiente y estructurada, lo cual es esencial para el análisis de datos en proyectos de investigación, negocios y diversas industrias.

R, como lenguaje de programación orientado al análisis de datos y la estadística, proporciona herramientas robustas para conectarse a diferentes tipos de bases de datos y llevar a cabo una amplia gama de análisis sobre los datos que se encuentran almacenados en ellas. En este sentido, una de las principales ventajas de R es su capacidad para interactuar de manera eficiente con bases de datos relacionales y no relacionales, lo cual amplía el rango de posibilidades para los analistas y científicos de datos.

El objetivo principal de este tema es familiarizar al estudiante con las herramientas y metodologías necesarias para conectar R con diferentes tipos de bases de datos, desde las tradicionales bases de datos relacionales como MySQL hasta las bases de datos NoSQL como MongoDB. Además, se abordará la manipulación de datos, su análisis y la generación de reportes a partir de la información obtenida de dichas bases de datos. Aprenderemos cómo realizar consultas a bases de datos desde R, extraer datos y llevar a cabo análisis complejos sin necesidad de exportar e importar datos manualmente.

A lo largo de este tema, se alcanzarán los siguientes objetivos específicos:

- ▶ Comprender la importancia de la conexión entre R y bases de datos para el análisis de datos eficiente y reproducible.
- ▶ Aprender a establecer conexiones entre R y diferentes tipos de bases de datos, incluyendo bases de datos relacionales (como MySQL) y NoSQL (como MongoDB).
- ▶ Explorar técnicas de manipulación y limpieza de datos a través de consultas a bases de datos desde R.
- ▶ Desarrollar habilidades para realizar análisis estadísticos sobre los datos extraídos y aprender a visualizar los resultados de forma efectiva.
- ▶ Introducir herramientas para la generación de reportes automatizados, permitiendo una comunicación clara y efectiva de los resultados del análisis.

En un entorno donde los datos son el principal activo, la capacidad de conectarse a bases de datos directamente desde R no solo mejora la eficiencia del proceso analítico, sino que también permite garantizar la reproducibilidad del trabajo, evitando la duplicación de datos y reduciendo el riesgo de errores en la gestión de información. El estudiante podrá explorar cómo R puede ser una herramienta versátil no solo para la manipulación y el análisis de datos, sino también para la gestión directa de estos, aprovechando la capacidad de almacenamiento y procesamiento de las bases de datos.

Este tema proporcionará una base sólida para entender las buenas prácticas en la gestión y el análisis de grandes volúmenes de datos, abarcando tanto los aspectos técnicos de la conexión y consulta de datos como los aspectos analíticos y de visualización que permiten extraer conclusiones útiles y accionables a partir de los mismos. Asimismo, se enfatizará en la importancia de la integridad y la calidad de los datos, aspectos clave para garantizar la validez de los resultados obtenidos.

8.2. Conexión de R con diferentes bases de datos

R proporciona diversas herramientas y paquetes para conectarse con diferentes tipos de bases de datos, lo cual facilita el trabajo con datos almacenados de manera eficiente. En este apartado, exploraremos las opciones disponibles para establecer conexiones con bases de datos relacionales y no relacionales, permitiendo a los usuarios acceder y trabajar directamente con los datos.

Paquetes de R para la conexión con bases de datos

- ▶ **DBI**: Este paquete proporciona una interfaz general para interactuar con bases de datos en R. Es utilizado para establecer la conexión con la base de datos y ejecutar consultas SQL de manera eficiente.
- ▶ **RMySQL** y **RMariaDB**: Paquetes específicos para conectarse a bases de datos MySQL o MariaDB. Permiten realizar consultas SQL y recuperar datos directamente en R.
- ▶ **RSQLite**: Permite la conexión con bases de datos SQLite, que son bases de datos ligeras que no requieren un servidor. Es útil para realizar pruebas y trabajar con bases de datos pequeñas.
- ▶ **mongolite**: Paquete utilizado para conectarse a bases de datos MongoDB, que es una base de datos NoSQL muy popular. Facilita la consulta y manipulación de documentos almacenados en formato JSON.

A través de estos paquetes, R permite realizar operaciones de lectura y escritura directamente sobre las bases de datos, lo cual optimiza el proceso de análisis al eliminar la necesidad de exportar datos manualmente.

Conexión a una base de datos SQLite

El siguiente ejemplo muestra cómo crear una base de datos SQLite local y conectarse desde R para insertar y consultar datos.

```
# Instalación y carga del paquete RSQLite
install.packages("RSQLite")
```

```
library(RSQLite)

# Creación de una conexión a una base de datos SQLite
con <- dbConnect(SQLite(), dbname = "mi_base_de_datos.sqlite")

# Creación de una tabla e inserción de datos
dbWriteTable(con, "empleados", data.frame(id = 1:3, nombre = c("Ana",
"Luis", "María"), salario = c(50000, 60000, 55000)))

# Consulta de los datos de la tabla
empleados <- dbReadTable(con, "empleados")
print(empleados)

# Cierre de la conexión
dbDisconnect(con)
```

Conexión a una base de datos MySQL

El siguiente ejemplo muestra cómo establecer una conexión con una base de datos MySQL remota y realizar una consulta.

```
# Instalación y carga del paquete RMySQL
install.packages("RMySQL")
library(RMySQL)

# Conexión a la base de datos MySQL *
con <- dbConnect(MySQL(), user = "usuario", password = "contraseña",
dbname = "mi_base_de_datos", host = "localhost")

# Consulta de una tabla
resultado <- dbGetQuery(con, "SELECT * FROM clientes")
print(resultado)

# Cierre de la conexión
dbDisconnect(con)

# * Configura la conexión para usar la base de datos de tu
conveniencia.
```

Conexión a una base de datos MongoDB

El siguiente ejemplo muestra cómo conectar R a MongoDB, insertar un documento JSON, recuperar los datos y mostrarlos por pantalla.

```
# Instalación y carga del paquete mongolite
install.packages("mongolite")
library(mongolite)

# Conexión a la base de datos MongoDB *
```

```

con <- mongo(collection = "mi_coleccion", db = "mi_base_de_datos",
url = "mongodb://localhost:27017")

# Inserción de un documento JSON en la colección
con$insert('{ "nombre": "Juan", "edad": 30, "profesion": "Ingeniero"
}')

# Recuperación de los datos de la colección
datos <- con$find()
print(datos)

# Cierre de la conexión
con$disconnect()

# * Configura la conexión para usar la base de datos de tu
conveniencia.

```

8.3. Conexión a MySQL

La conexión de R con bases de datos MySQL se lleva a cabo mediante el uso del paquete RMySQL, que permite establecer conexiones con bases de datos relacionales MySQL y ejecutar consultas SQL para realizar operaciones CRUD. MySQL es una de las bases de datos relacionales más populares y se utiliza ampliamente para almacenar datos estructurados que necesitan ser accedidos y manipulados de manera eficiente.

Conexión a MySQL

Para conectarse a MySQL desde R, utilizamos la función `dbConnect()` del paquete RMySQL, donde especificamos el usuario, la contraseña, el nombre de la base de datos y el host. Por ejemplo:

```

con <- dbConnect(MySQL(), user = "usuario", password = "contraseña", dbname
= "mi_base_de_datos", host = "localhost")

```

Esto crea una conexión con la base de datos `mi_base_de_datos` alojada en el servidor local (`localhost`). Una vez establecida la conexión, se pueden realizar diversas operaciones sobre la base de datos, tales como la inserción, actualización, consulta y borrado de datos.

Operaciones CRUD en MySQL desde R

1. **Inserción de Datos:** Para insertar datos en una tabla, utilizamos el método `dbWriteTable()` o `dbSendQuery()`. Por ejemplo, para insertar datos directamente en una tabla llamada `clientes`:

```
dbWriteTable(con, "clientes", data.frame(id = 1:3, nombre = c("Ana", "Luis",  
"María"), edad = c(28, 35, 42)), append = TRUE)
```

También se puede insertar utilizando una consulta SQL:

```
dbSendQuery(con, "INSERT INTO clientes (id, nombre, edad) VALUES (4,  
'Carlos', 30)")
```

2. **Consulta de Datos:** Para recuperar datos de una tabla, utilizamos el método `dbGetQuery()`. Podemos realizar consultas SQL para filtrar los resultados:

```
resultado <- dbGetQuery(con, "SELECT * FROM clientes WHERE edad > 30")  
print(resultado)
```

3. **Actualización de Datos:** Para actualizar registros existentes en una tabla, utilizamos el método `dbSendQuery()` con una consulta SQL de actualización:

```
dbSendQuery(con, "UPDATE clientes SET edad = 36 WHERE nombre = 'Luis'")
```

4. **Borrado de Datos:** Para borrar registros de una tabla, también utilizamos `dbSendQuery()` con una consulta SQL de borrado:

```
dbSendQuery(con, "DELETE FROM clientes WHERE nombre = 'Ana'")
```

Reto: conectar R a una base de datos MySQL, insertar varios registros en una tabla, realizar consultas y análisis, y generar visualizaciones.

```

# Instalación y carga del paquete RMySQL
install.packages("RMySQL")
install.packages("ggplot2")
library(RMySQL)
library(ggplot2)

# Conexión a la base de datos MySQL
con <- dbConnect(MySQL(), user = "usuario", password = "contraseña", dbname
= "mi_base_de_datos", host = "localhost")

# Inserción de varios registros en la tabla clientes
dbWriteTable(con, "clientes", data.frame(id = 1:5, nombre = c("Carlos",
"Ana", "Luis", "María", "Jorge"), edad = c(25, 30, 35, 40, 45)), append =
TRUE)

# Recuperación de todos los registros de la tabla clientes
datos <- dbGetQuery(con, "SELECT * FROM clientes")

# Análisis de los datos
print(datos)

# Visualización: Histograma de edades
ggplot(datos, aes(x = edad)) +
  geom_histogram(binwidth = 5, fill = "blue", color = "black") +
  labs(title = "Distribución de Edades", x = "Edad", y = "Frecuencia")

# Visualización: Gráfico de barras por nombre
ggplot(datos, aes(x = nombre)) +
  geom_bar(fill = "purple", color = "black") +
  labs(title = "Cantidad de Personas por Nombre", x = "Nombre", y =
"Frecuencia")

# Visualización: Gráfico de dispersión edad vs nombre
ggplot(datos, aes(x = edad, y = nombre)) +
  geom_point(color = "red") +
  labs(title = "Edad vs Nombre", x = "Edad", y = "Nombre")

# Cierre de la conexión

```

```
dbDisconnect(con)
```

En este ejercicio, primero establecemos una conexión con una base de datos MySQL y luego insertamos varios registros en la tabla clientes. A continuación, recuperamos todos los registros y realizamos un análisis básico, mostrando un histograma de las edades de las personas almacenadas en la base de datos y otras visualizaciones. Estos gráficos permiten comprender mejor la distribución y las características de los datos almacenados en la base de datos MySQL

8.4. Conexión a MongoDB

Para conectarse a MongoDB desde R, utilizamos la función `mongo()` del paquete `mongolite`, donde especificamos la colección y la base de datos a la que queremos conectarnos, así como la URL del servidor. Por ejemplo:

```
con <- mongo(collection = "mi_coleccion", db = "mi_base_de_datos", url =  
"mongodb://localhost:27017")
```

Esto crea una conexión con la base de datos `mi_base_de_datos` y la colección `mi_coleccion` en un servidor MongoDB que se ejecuta localmente en el puerto 27017.

Operaciones CRUD en MongoDB desde R

1. **Inserción de Documentos:** Para insertar documentos JSON en una colección de MongoDB, se utiliza el método `insert()`. Por ejemplo:

```
con$insert('{ "nombre": "Ana", "edad": 28, "profesion": "Diseñadora" }')
```

También es posible insertar múltiples documentos a la vez utilizando un vector de caracteres JSON.

2. **Consulta de Documentos:** Para recuperar documentos de la colección, utilizamos el método `find()`. Podemos especificar criterios de búsqueda para filtrar los resultados. Por ejemplo:

```
datos <- con$find('{ "profesion": "Ingeniero" }')
print(datos)
```

3. **Actualización de Documentos:** Para actualizar un documento existente, se utiliza el método `update()`, donde especificamos un filtro para encontrar el documento a actualizar y los nuevos valores. Por ejemplo:

```
con$update('{ "nombre": "Juan" }', '{ "$set": { "edad": 31 } }')
```

4. **Borrado de Documentos:** Para borrar documentos de la colección, se utiliza el método `remove()`. Por ejemplo, para borrar todos los documentos donde el nombre sea "Ana":

```
con$remove('{ "nombre": "Ana" }')
```

Reto propuesto: Recuperar varios documentos de MongoDB y realizar un análisis estadístico de los datos, incluyendo visualizaciones.

```
# Instalación y carga del paquete mongolite
install.packages("mongolite")
install.packages("ggplot2")
library(mongolite)
library(ggplot2)

# Conexión a la base de datos MongoDB
con <- mongo(collection = "mi_coleccion", db = "mi_base_de_datos", url =
"mongodb://localhost:27017")

# Inserción de varios documentos JSON
con$insert('[
  { "nombre": "Carlos", "edad": 25, "profesion": "Analista" },
  { "nombre": "María", "edad": 29, "profesion": "Data Scientist" },
  { "nombre": "Luis", "edad": 35, "profesion": "Ingeniero" }
]')

# Recuperación de todos los documentos de la colección
datos <- con$find()
```

```

# Análisis de los datos
print(datos)

# Visualización: Histograma de edades
ggplot(datos, aes(x = edad)) +
  geom_histogram(binwidth = 5, fill = "blue", color = "black") +
  labs(title = "Distribución de Edades", x = "Edad", y = "Frecuencia")
# Visualización: Gráfico de barras por profesión
ggplot(datos, aes(x = profesion)) +
  geom_bar(fill = "green", color = "black") +
  labs(title = "Cantidad de Personas por Profesión", x = "Profesión", y =
"Frecuencia")

# Visualización: Gráfico de dispersión edad vs profesión
ggplot(datos, aes(x = edad, y = profesion)) +
  geom_point(color = "red") +
  labs(title = "Edad vs Profesión", x = "Edad", y = "Profesión")

# Cierre de la conexión
con$disconnect()

```

En este ejercicio, primero conectamos a una base de datos MongoDB y luego insertamos varios documentos. A continuación, recuperamos todos los documentos y realizamos un análisis básico, mostrando un histograma de las edades de las personas almacenadas en la base de datos. Esta visualización ayuda a entender mejor la distribución de los datos y a obtener insights de manera gráfica.

Para una mejor comprensión de los datos, se realizaron las siguientes visualizaciones:

Histograma de Edades

- Este gráfico muestra la distribución de las edades de las personas en el dataset. La mayoría de los individuos se encuentran en el rango de edad de 25 a 35 años.

```

# Visualización: Histograma de edades
ggplot(datos, aes(x = edad)) +
  geom_histogram(binwidth = 5, fill = "blue", color = "black") +
  labs(title = "Distribución de Edades", x = "Edad", y = "Frecuencia")

```

Gráfico de Barras por Profesión

- Este gráfico muestra la cantidad de personas por profesión. Podemos observar que la profesión más común es "Ingeniero", seguida por "Data Scientist".

Visualización: Gráfico de barras por profesión

```
ggplot(datos, aes(x = profesion)) +  
  geom_bar(fill = "green", color = "black") +  
  labs(title = "Cantidad de Personas por Profesión", x = "Profesión", y =  
"Frecuencia")
```

Gráfico de Dispersión Edad vs Profesión

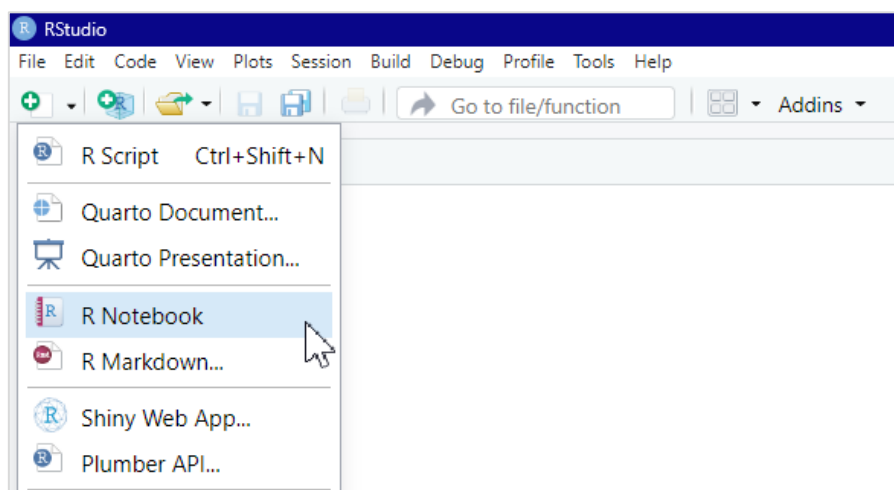
- Este gráfico permite visualizar la relación entre la edad y la profesión de las personas en el dataset.

Visualización: Gráfico de dispersión edad vs profesión

```
ggplot(datos, aes(x = edad, y = profesion)) +  
  geom_point(color = "red") +  
  labs(title = "Edad vs Profesión", x = "Edad", y = "Profesión")
```

Ejemplo de Informe

A continuación, se presenta un ejemplo de informe utilizando un dataset más amplio recuperado de una base de datos MongoDB. Este informe incluye un análisis detallado de los datos y la generación de visualizaciones para comunicar los resultados.



Figuras 1: Selección de recursos Notebook en R para crear los informes en Markdown y R.

El ejemplo a continuación es un breve informe que muestra información en Markdown y código R para su ejecución.

Informe de Análisis de Datos - Conexión con MongoDB

Introducción

Este informe tiene como objetivo analizar un conjunto de datos almacenado en una base de datos MongoDB, utilizando R para recuperar, analizar y visualizar la información. La base de datos contiene información sobre varias personas, incluyendo su edad, profesión y otras características relevantes. A través de este análisis, se busca identificar patrones y tendencias que puedan ser útiles para la toma de decisiones.

Conexión y Extracción de Datos

Utilizamos el paquete `mongolite` para conectarnos a MongoDB y extraer los datos necesarios para el análisis. A continuación, se muestra el código utilizado para la conexión y la extracción de los datos:

```
```{r}
Instalación y carga del paquete mongolite
install.packages("mongolite")
install.packages("ggplot2")
library(mongolite)
library(ggplot2)

Conexión a la base de datos MongoDB
```

```

con <- mongo(collection = "mi_coleccion", db = "mi_base_de_datos", url =
"mongodb://localhost:27017")

Recuperación de todos los documentos de la colección
datos <- con$find()

Análisis descriptivo
summary(datos$edad)
table(datos$profesion)

con$disconnect()
Recuerda disponer de una base de datos que cumpla los criterios del
ejercicio.
```

```

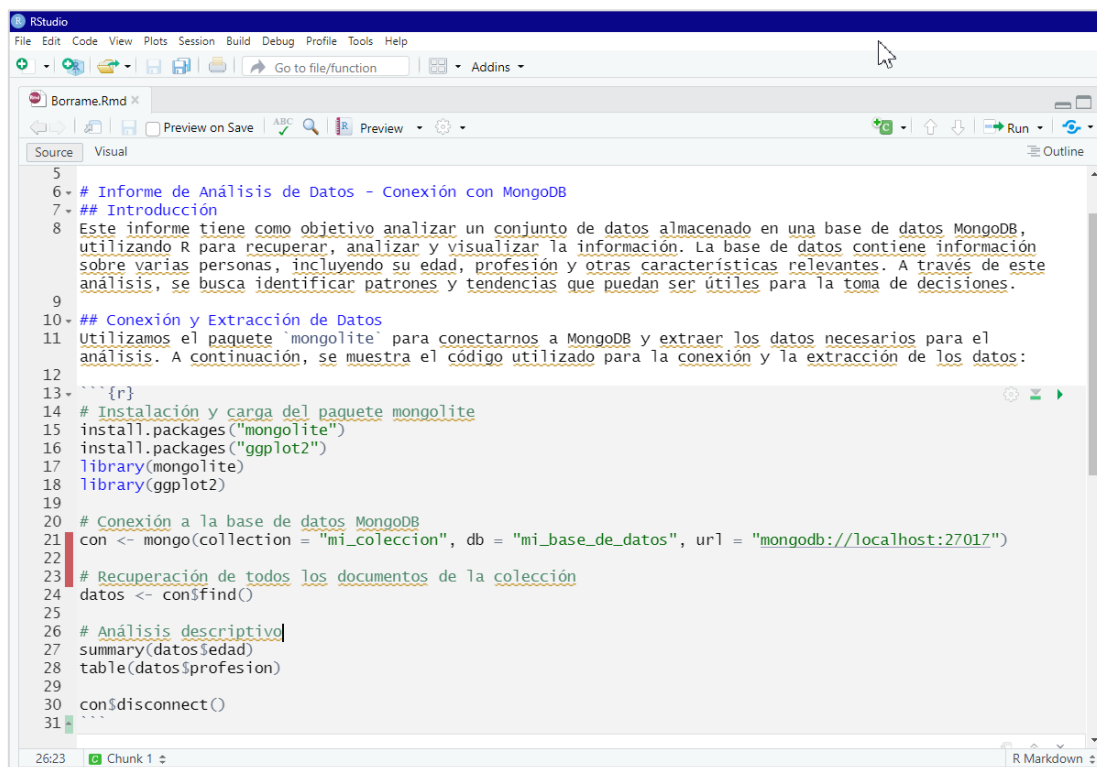
El dataset recuperado contiene información sobre la edad y la profesión de varias personas. Realizamos un análisis descriptivo de las variables principales para comprender mejor la distribución y características de los datos.

- ▶ Resumen de Edades: Se calculó el promedio, la mediana y la desviación estándar de las edades.
- ▶ Distribución de Profesiones: Se identificaron las profesiones más comunes en el dataset.

```

# Análisis descriptivo
summary(datos$edad)
table(datos$profesion)

```



Figuras 2: Ejemplo del informe en Markdown y R.

A partir del análisis realizado, se puede concluir que la mayoría de las personas en el dataset se encuentran en un rango de edad joven, principalmente entre 25 y 35 años, y que las profesiones predominantes son "Ingeniero" y "Data Scientist". Estos hallazgos pueden ser útiles para enfocar estrategias de capacitación y desarrollo profesional hacia los perfiles más comunes en el grupo de estudio.

8.5. Manipulación y Análisis de Datos

En este apartado, nos enfocaremos en la manipulación y análisis de datos utilizando R para construir un Dashboard interactivo que cuente una historia significativa a partir de un conjunto de datos recuperados de una base de datos. Un Dashboard permite visualizar patrones, tendencias y resúmenes clave, facilitando la toma de decisiones basada en datos. R, gracias a su integración con paquetes potentes como *ggplot2*, *shiny*, y *dplyr*, permite crear Dashboard de análisis dinámicos y eficaces.

Trabajaremos con una base de datos SQLite, demostrando cómo crear tablas, insertar datos desde un dataset existente, realizar transformaciones y luego generar gráficos para contar una historia completa. Utilizaremos el paquete RSQLite y otros componentes del ecosistema Tidyverse para este propósito.

Paso 1: Creación de una Base de Datos SQLite y Población de Datos

Primero, conectaremos a una base de datos SQLite y crearemos una tabla a partir de un dataset conocido, *mtcars*, que viene incluido en R y contiene información sobre diferentes modelos de automóviles.

```
# Instalación y carga de paquetes necesarios
install.packages("RSQLite")
install.packages("dplyr")
install.packages("ggplot2")
library(RSQLite)
library(dplyr)
library(ggplot2)

# Creación de una conexión a una base de datos SQLite
con <- dbConnect(SQLite(), dbname = "mi_base_de_datos.sqlite")

# Creación de una tabla 'autos' e inserción de datos desde el dataset
'mtcars'
dbWriteTable(con, "autos", mtcars, overwrite = TRUE)

# Consulta de los datos de la tabla
autos <- dbReadTable(con, "autos")
print(autos)
```

En este paso, hemos creado una base de datos SQLite y la hemos llenado con datos del dataset *mtcars*. Esto nos permitirá trabajar con una base de datos real para realizar manipulaciones y análisis.

Paso 2: Recuperación y Transformación de Datos

A continuación, recuperaremos los datos de la base de datos y aplicaremos algunas transformaciones usando dplyr. Queremos añadir una nueva columna que clasifique a los autos según su eficiencia de combustible (mpg).

```
# Recuperación de los datos y conexión
autos <- tbl(con, "autos") %>% collect()

# Clasificación de eficiencia de combustible
autos <- autos %>%
  mutate(eficiencia = case_when(
    mpg > 20 ~ "Alta",
    mpg > 15 ~ "Media",
    TRUE ~ "Baja"
  ))
print(autos)
```

Hemos añadido una columna llamada eficiencia que clasifica cada auto según su rendimiento de combustible (Alta, Media, Baja). Esta clasificación nos ayudará en los siguientes pasos para contar una historia más clara sobre los datos.

Paso 3: Visualización de los Datos

Ahora que hemos transformado los datos, construiremos varios gráficos para explorar las características de los automóviles.

Histograma del Rendimiento de Combustible (mpg)

```
ggplot(autos, aes(x = mpg)) +
  geom_histogram(binwidth = 3, fill = "blue", color = "black") +
  labs(title = "Distribución del Rendimiento de Combustible (mpg)", x =
"Millas por Galón (mpg)", y = "Frecuencia")
```

Gráfico de Barras de la Eficiencia de Combustible

```
ggplot(autos, aes(x = eficiencia)) +
  geom_bar(fill = "orange", color = "black") +
  labs(title = "Distribución de la Eficiencia de Combustible", x =
"Eficiencia", y = "Frecuencia")
```

Gráfico de Dispersión entre hp (Caballos de Fuerza) y mpg (Rendimiento)

```
ggplot(autos, aes(x = hp, y = mpg)) +
  geom_point(color = "red") +
  labs(title = "Relación entre Caballos de Fuerza y Rendimiento", x =
"Caballos de Fuerza (hp)", y = "Millas por Galón (mpg)")
```

Gráfico de Caja (Boxplot) para mpg por Número de Cilindros (cyl)

```
ggplot(autos, aes(x = as.factor(cyl), y = mpg)) +  
  geom_boxplot(fill = "green", color = "black") +  
  labs(title = "Rendimiento de Combustible por Número de Cilindros", x =  
"Número de Cilindros", y = "Millas por Galón (mpg)")
```

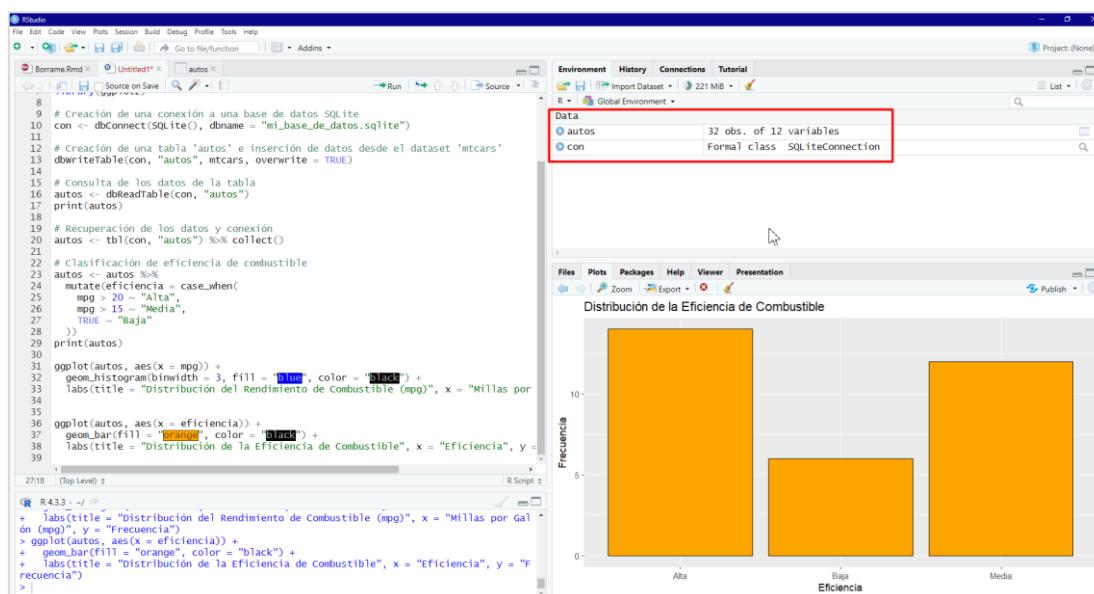
Gráfico de Línea de Promedio de mpg por Peso (wt)

```
autos %>%  
  group_by(wt) %>%  
  summarise(mpg_promedio = mean(mpg)) %>%  
  ggplot(aes(x = wt, y = mpg_promedio)) +  
  geom_line(color = "purple") +  
  labs(title = "Rendimiento Promedio de Combustible por Peso", x = "Peso  
(1000 lbs)", y = "Millas por Galón Promedio")
```

Paso 4: Interpretación y Conclusión

Los gráficos creados nos permiten comprender mejor las características del dataset mtcars y sus relaciones:

- ▶ **Histograma de mpg:** Nos muestra cómo se distribuye el rendimiento de combustible entre los autos, identificando qué rangos son más comunes.
- ▶ **Gráfico de Eficiencia de Combustible:** Nos proporciona una vista clara de cuántos autos caen en cada categoría de eficiencia.
- ▶ **Relación hp vs mpg:** El gráfico de dispersión indica una correlación negativa, lo cual sugiere que a mayor cantidad de caballos de fuerza, menor es el rendimiento de combustible.
- ▶ **Boxplot de mpg por cyl:** Nos permite observar cómo el rendimiento varía según el número de cilindros, proporcionando una visión clara de la eficiencia según el tamaño del motor.
- ▶ **Promedio de mpg por Peso:** El gráfico de línea muestra cómo el peso del auto influye en su rendimiento, indicando que los autos más pesados tienden a tener menor eficiencia de combustible.



Figuras 3: Ejercicio de SQLite: el ejercicio pretende mostrar como la fuente de datos es una BD y no una tabla de librería.

8.6. Visualización de Datos y Reporting

La visualización de datos es fundamental para comprender patrones y relaciones dentro de un dataset. Este apartado se enfoca en el proceso de visualización de datos y la generación de reportes utilizando R como herramienta clave. Aprovecharemos un dataset almacenado en una URL pública, cargaremos el dataset en una base de datos SQLite, realizaremos transformaciones sobre los datos y, finalmente, crearemos gráficos interactivos para construir un Dashboard utilizando Shiny, una potente librería de R que facilita la creación de aplicaciones web interactivas. A continuación, se detalla cada paso del proceso, destacando las ventajas de utilizar R para este propósito.

Paso 1: Carga del Dataset

En primer lugar, se debe cargar el dataset desde una URL pública. En R, podemos utilizar la función `read.csv()` para leer el archivo directamente desde la URL. El dataset está alojado en <https://data.cityofnewyork.us/api/views/c3uy-2p5r/rows.csv>.

```
# Cargar librerías necesarias
library(RSQLite)
library(DBI)

# Leer el dataset desde la URL
dataset_url <- "https://data.cityofnewyork.us/api/views/c3uy-2p5r/rows.csv"

data <- read.csv(dataset_url)
```

Explora el dataset y revisa los nombres de las columnas.

Una vez cargado el dataset en R, podemos visualizar sus primeras filas para comprender su estructura y contenido. Esto nos permitirá identificar las columnas y los tipos de datos disponibles.

```
# Mostrar las primeras filas del dataset
head(data)
```

Paso 2: Creación de la Base de Datos SQLite y Almacenamiento del Dataset

El siguiente paso consiste en crear una base de datos SQLite local y almacenar el dataset en ella para facilitar su manipulación. Esto es particularmente útil cuando se trabajan con grandes volúmenes de datos, ya que permite realizar consultas SQL para filtrar y transformar los datos de manera eficiente.

```
# Crear conexión con la base de datos SQLite
con <- dbConnect(RSQLite::SQLite(), "data_analysis.db")

# Guardar el dataset en SQLite
dbWriteTable(con, "data_table", data, overwrite = TRUE)
```

Con el dataset almacenado en SQLite, podemos realizar consultas SQL para seleccionar, filtrar o transformar los datos de acuerdo con nuestras necesidades.

Paso 3: Transformación de los Datos

Dado el dataset proporcionado, vamos a realizar varias transformaciones y cálculos adicionales relevantes.

Cantidad Total de Medidas por Tipo de Indicador

Queremos calcular la cantidad total de medidas por tipo de indicador (Indicator ID) y agrupar los datos por dicho indicador para analizar cuáles son los más comunes.

```
# Realizar una consulta para transformar los datos
query <- "SELECT `Indicator ID`, COUNT(*) AS total_measures FROM data_table
GROUP BY `Indicator ID` ORDER BY total_measures DESC"
transformed_data <- dbGetQuery(con, query)
```

Esta consulta nos permite obtener un resumen del número total de registros por cada tipo de indicador, lo cual es útil para entender la distribución de las medidas reportadas.

Promedio de Valores de Medidas por Lugar Geográfico

A continuación, realizaremos una consulta para calcular el promedio del valor de la medida (Data Value) por cada lugar geográfico (Geo Place Name).

```
# Consulta para obtener el promedio del valor de la medida por lugar
geográfico
query_avg <- "SELECT `Geo Place Name`, AVG(`Data Value`) AS avg_value FROM
data_table GROUP BY `Geo Place Name`"
avg_data <- dbGetQuery(con, query_avg)
```

Esta consulta nos ayuda a identificar el valor promedio de cada indicador según la ubicación geográfica.

Transformación de Campos Categóricos

Transformaremos algunos campos categóricos para facilitar el análisis. En este caso, podemos convertir el Indicator ID en una variable factor para una mejor manipulación en R.

```
# Transformar el campo Indicator ID en un factor
```

```
data$`Indicator ID` <- as.factor(data$`Indicator ID`)
```

Filtrado de Datos por Periodo de Tiempo

Supongamos que queremos analizar los datos de un periodo específico. Podemos filtrar los datos utilizando una consulta SQL para seleccionar solo aquellos registros dentro del período deseado.

```
# Filtrar datos por un rango de fechas específico
query_date <- "SELECT * FROM data_table WHERE Start_Date BETWEEN '2015-01-01' AND '2015-12-31'"
filtered_data <- dbGetQuery(con, query_date)
```

Este filtrado permite centrarse en los datos de un año específico, lo cual es útil para realizar análisis temporales.

Cálculo de Medidas por Tipo de Lugar Geográfico

Para entender la distribución geográfica de las medidas, podemos agrupar los datos por el tipo de lugar (Geo Type Name) y calcular el total de medidas por cada uno.

```
# Realizar una consulta para calcular la cantidad de medidas por tipo de lugar
query_geo <- "SELECT `Geo Type Name`, COUNT(*) AS total_measures FROM data_table GROUP BY `Geo Type Name` ORDER BY total_measures DESC"
geo_data <- dbGetQuery(con, query_geo)
```

Esto nos proporciona una visión clara de qué tipos de lugares tienen la mayor cantidad de medidas reportadas.

Creación de Nuevas Variables

Podemos crear una nueva variable que indique si un valor de medida supera un umbral determinado, como por ejemplo un valor mayor a 1. Esto nos permitirá identificar rápidamente valores inusuales o de interés.

```
# Crear una nueva columna indicando si el valor de la medida es mayor a 1
data$high_value <- ifelse(data$`Data Value` > 1, "Sí", "No")
```

Esta nueva columna nos ayudará a identificar rápidamente los valores de medida que exceden un determinado umbral.

Paso 4: Visualización de los Datos

R ofrece una amplia variedad de librerías para la visualización de datos, tales como ggplot2 y plotly. En este caso, utilizaremos ggplot2 para generar gráficos detallados y fáciles de interpretar.

Ejemplo de Gráfico de Barras

Un gráfico de barras es ideal para mostrar la cantidad de medidas por tipo de indicador.

```
# Cargar librería ggplot2
library(ggplot2)

# Crear gráfico de barras
ggplot(transformed_data, aes(x = reorder(`Indicator ID`, -total_measures),
y = total_measures)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  theme_minimal() +
  labs(title = "Cantidad de Medidas por Tipo de Indicador", x = "Tipo de
Indicador", y = "Cantidad de Medidas") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Ejemplo de Gráfico de Mapa

Dado que el dataset contiene información geográfica (nombre del lugar), podemos visualizar las medidas en un mapa para entender mejor su distribución geográfica en la ciudad de Nueva York (en el caso de que el dataset tuviese la información de Longitud y Latitud).

```
# Cargar librerías necesarias
library(leaflet)

# Crear un mapa interactivo con la ubicación de las medidas (asumiendo
latitud y longitud disponibles)
leaflet(data) %>%
  addTiles() %>%
  addCircles(lng = ~LONGITUDE, lat = ~LATITUDE, popup = ~`Geo Place Name`,
radius = 50, color = "red", fillOpacity = 0.5) %>%
  setView(lng = -73.935242, lat = 40.730610, zoom = 10)
```

Paso 5: Creación del Dashboard con Shiny

Finalmente, utilizamos Shiny para construir un dashboard interactivo que permita a los usuarios explorar los datos de manera dinámica. Shiny es una de las librerías más poderosas de R para crear aplicaciones web interactivas, y nos permitirá combinar todos los gráficos generados en una sola interfaz.

Código del Dashboard

```
# Cargar librería Shiny
library(shiny)

# Definir la interfaz de usuario
ui <- fluidPage(
  titlePanel("Dashboard de Análisis de Medidas Ambientales en NYC"),
  sidebarLayout(
    sidebarPanel(
      selectInput("geo_place", "Seleccione un lugar geográfico:", choices =
unique(avg_data$`Geo Place Name`))
    ),
    mainPanel(
      plotOutput("barPlot"),
      plotOutput("avgPlot")
    )
  )
)

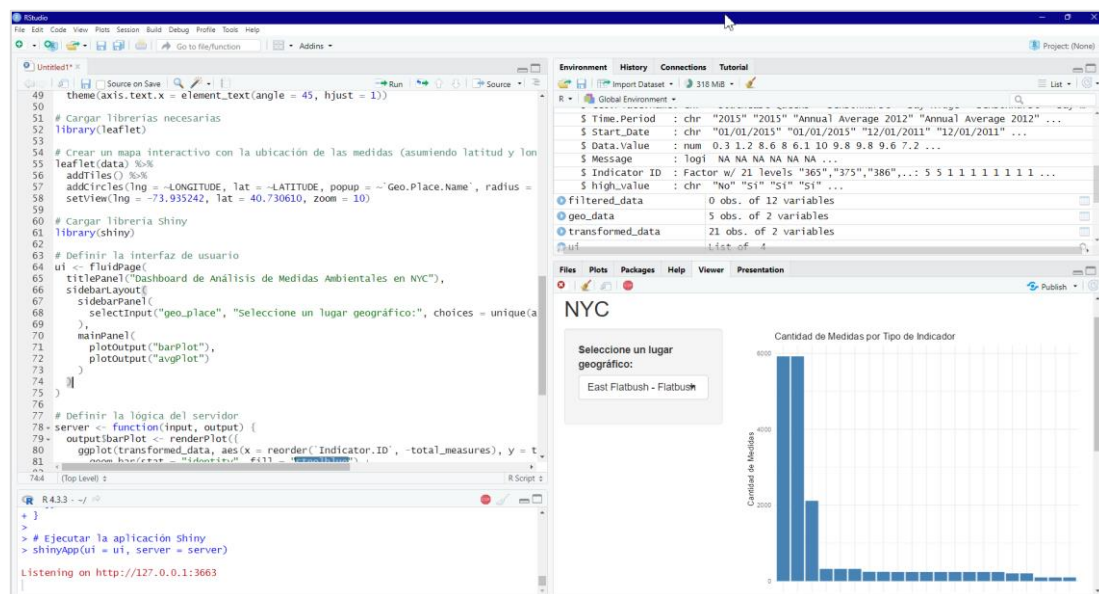
# Definir la lógica del servidor
server <- function(input, output) {
  output$barPlot <- renderPlot({
    ggplot(transformed_data, aes(x = reorder(`Indicator ID`,
total_measures), y = total_measures)) +
      geom_bar(stat = "identity", fill = "steelblue") +
      theme_minimal() +
      labs(title = "Cantidad de Medidas por Tipo de Indicador", x = "Tipo
de Indicador", y = "Cantidad de Medidas")
  })

  output$avgPlot <- renderPlot({
    filtered_avg_data <- avg_data[avg_data$`Geo Place Name` ==
input$geo_place, ]
    ggplot(filtered_avg_data, aes(x = `Geo Place Name`, y = avg_value)) +
      geom_bar(stat = "identity", fill = "darkgreen") +
      theme_minimal() +
      labs(title = "Promedio del Valor de Medidas por Lugar Geográfico", x
= "Lugar Geográfico", y = "Valor Promedio de Medida")
  })
}

# Ejecutar la aplicación Shiny
shinyApp(ui = ui, server = server)
```

Ventajas de Utilizar R para Visualización de Datos y Reporting

- Flexibilidad: R ofrece una gran flexibilidad para el análisis y la visualización de datos gracias a su amplia gama de librerías, como ggplot2, leaflet y plotly.
- Integración con Bases de Datos: La capacidad de conectar R con bases de datos como SQLite permite manejar grandes volúmenes de datos de forma eficiente y realizar transformaciones complejas con facilidad.
- Interactividad: Con Shiny, es posible crear dashboards interactivos que permiten a los usuarios explorar los datos de manera dinámica, lo cual es ideal para la toma de decisiones informada.
- Comunidad Activa: R cuenta con una comunidad activa y una gran cantidad de recursos en línea, lo cual facilita la resolución de problemas y la implementación de nuevas ideas.



Figuras 4: Ejercicio de visualización interactiva de datos.

R proporciona un entorno integral para el análisis, transformación, visualización y presentación de datos, lo cual lo hace ideal para la generación de reportes detallados y Dashboard interactivos.

8.7. Referencias bibliográficas

Xie, Y., Dervieux, C. y Riederer, E. (2020). *R Markdown Cookbook*. First edition, C&H/CRC Press, 2021.

<https://bookdown.org/yihui/rmarkdown-cookbook/rmarkdown-process.html>

Wickham, H. and Grolemund, G. (2019). *R para Ciencia de Datos*.

<https://es.r4ds.hadley.nz/>

8.8. Cuaderno de ejercicios

Ejercicio 1

Copia el código a continuación en un Notebook de RStudio y ejecútalo.

```
---
title: "Shiny - Ejemplos Parte 1"
author: ""
date: "14/04/2024"
output: html_document
runtime: shiny
---

```{r}
library(shiny)
```

# 1. Creación de la aplicación

## Crea una app en RStudio

El proceso es el siguiente:

>- ``File`` > ``New File`` > ``Shiny Web App`` >
>- Rellena los campos: ``Application name`` y ``directory``

* Esta acción creará una carpeta en el directorio de trabajo siguiendo el nombre indicado previamente.

* Dentro de la carpeta estará el fichero app.R que contiene un código inicial de ejemplo, con la estructura que necesita la aplicación.

* Rstudio detecta que es una aplicación shiny y activa el boton "Run app" en la ventana de scripts (en el marco, zona superior derecha)

* El ejemplo es plenamente funcional, probarlo dándole a "Run app". Probarlo tambien en navegador.

## Aplicación base
```

```

```{r, echo=FALSE}
library(shiny)

ui <- fluidPage()

server <- function(input, output) {}

shinyApp(server = server, ui = ui)
```

## Una entrada y cero salidas
```{r, echo=FALSE}
library(shiny)
ui <- fluidPage(
 sliderInput(inputId = "num",
 label = "Selecciona un numero",
 value = 25, min = 10, max = 100)
)

server <- function(input, output) {}

shinyApp(server = server, ui = ui)
```

## Dos entradas y una salida
```{r}
library(shiny)
ui <- fluidPage(
 sliderInput(inputId = "num1",
 label = "Selecciona un numero",
 value = 25, min = 1, max = 100),
 # sliderInput(inputId = "num2",
 # label = "Selecciona un numero",
 # value = 25, min = 1, max = 100),
 textInput(inputId = "title",
 label = "Escribe un titulo",
 value = "Histograma de valores aleatorios Normales"),

 plotOutput("histo")
)
server <- function(input, output) {
 set.seed(1234)
 output$histo <- renderPlot({
 hist(rnorm(input$num1), main = input$title)
 })
}

shinyApp(ui = ui, server = server)
```

## Dos entradas y dos salida
```{r}
library(shiny)

ui <- fluidPage(
 sliderInput(inputId = "num",

```

```

 label = "Selecciona un numero",
 value = 25, min = 1, max = 100),
 plotOutput("hist"),
 verbatimTextOutput("stats")
)

server <- function(input, output) {
 output$hist <- renderPlot({
 hist(rnorm(input$num))
 })
 output$stats <- renderPrint({
 summary(rnorm(input$num))
 })
}

shinyApp(ui = ui, server = server)
```

```

Ejercicio 2

Copia el código a continuación en un Notebook de RStudio y ejecútalo.

```

---
title: "Shiny - Ejemplos Parte 2"
author: ""
date: "14/04/2024"
output: html_document
runtime: shiny
---

```{r}
library(shiny)

2.- Control de la reactividad

Crear una expresión reactiva

`reactive` envuelve una expresión normal para crear una expresión reactiva.
Conceptualmente, es una expresión cuyo resultado cambiará con el tiempo.

```{r}
library(shiny)

ui <- fluidPage(
  sliderInput(inputId = "num",
    label = "Selecciona un numero",
    value = 25, min = 1, max = 100),
  plotOutput("hist"),
  verbatimTextOutput("stats"),
  verbatimTextOutput("len")
)

server <- function(input, output) {

  data <- reactive({

```

```

# numero <- (input$num *10 )/27
# rnorm(numero)
rnorm(input$num *10)

#rnorm(input$num)
# Aquí pueden haber muchas más operaciones...

})

output$hist <- renderPlot({
  hist(data())
})
output$stats <- renderPrint({
  summary(data())
})
output$len <- renderPrint({
  #length(data())
  data()
})
}

shinyApp(ui = ui, server = server)
```


Crear un entorno que no sea reactivo

`isolate` causa que no se dispare la reacción reactiva dentro de un entorno reactivo pero permite acceder a los valores.


```{r}



```

library(shiny)
ui <- fluidPage(
 sliderInput(inputId = "num",
 label = "Selecciona un numero",
 value = 25, min = 1, max = 100),
 textInput(inputId = "title",
 label = "Escribe un titulo",
 value = "Histograma de valores aleatorios Normales"),
 plotOutput("hist"),
 verbatimTextOutput("stats")
)

server <- function(input, output) {

 output$hist <- renderPlot({
 hist(rnorm(isolate(input$num)), main = input$title)
 })
 output$stats <- renderPrint({
 as.character(input$num)
 })
}

shinyApp(ui = ui, server = server)
```



## Botón de acción


```


```

``actionButton`` crea un botón o enlace cuyo valor inicial es cero, pero que se incrementa cada vez que se pulsa.

```
```{r}
library(shiny)

ui <- fluidPage(
  actionButton(inputId = "clicks", label = "Dame caña!"),
  verbatimTextOutput("nclick")
)

server <- function(input, output) {

  observeEvent(input$clicks, {
    print(as.numeric(input$clicks))
  })
  # output$nclick <- renderPrint({
  #   as.character(input$clicks)
  # })
}

shinyApp(ui = ui, server = server)
```
```

## Observar eventos reactivos

``observeEvent`` responde a evento reactivos de entradas, valores, expresiones.

```
```{r}
library(shiny)

ui <- fluidPage(
  sliderInput(inputId = "num",
    label = "Selecciona un numero",
    min = 1, max = 100, value = 25),
  actionButton(inputId = "boton1",
    label = "ACTUALIZAR")
)

server <- function(input, output) {

  # observe responds to the print button
  # but not the slider
  observeEvent(input$boton1, {
    print(as.numeric(input$num))
  })
}

shinyApp(ui = ui, server = server)
```
```

## Reaccionar a eventos reactivos

``eventReactive`` responde (y permite devolver/calcular valores) a evento reactivos de entradas, valores, expresiones.

```

```{r}

library(shiny)

ui <- fluidPage(
  sliderInput(inputId = "num",
    label = "Selecciona un numero",
    value = 25, min = 1, max = 100),
  actionButton(inputId = "go", label = "Update"),
  plotOutput("hist")
)

server <- function(input, output) {
  data <- eventReactive(input$go, {
    rnorm(input$num)
  })

  output$hist <- renderPlot({
    hist(data())
  })
}

shinyApp(ui = ui, server = server)
```

```

## ## Valores reactivos

Esta función devuelve un objeto para almacenar valores reactivos. Es similar a una lista, pero con capacidades especiales para programación reactiva. Cuando se lee un valor de él, la expresión reactiva que llama toma una dependencia reactiva de ese valor, y cuando se escribe en él, notifica cualquier función reactiva que dependa de ese valor. Los valores tomados del objeto `reactiveValues` son reactivos, pero el objeto `reactiveValues` en sí no lo es.

```

```{r}
library(shiny)

ui <- fluidPage(
  actionButton(inputId = "norm", label = "Normal"),
  actionButton(inputId = "unif", label = "Uniform"),
  sliderInput(inputId = "num",
    label = "Selecciona un numero",
    value = 25,
    min = 1,
    max = 100),
  plotOutput("hist"),
  verbatimTextOutput("texto")
)

server <- function(input, output) {

  numero <- reactive({input$num})

  rv <- reactiveValues(data = rnorm(100),
    titulo="Distribucion normal",
    total=19885)

```

```

observeEvent(input$norm, {
  rv$data <- rnorm(numero())
  rv$titulo <- "Ahora es normal"})
observeEvent(input$unif, {
  rv$data <- runif(numero())
  rv$titulo <- "Ahora es uniforme"})

output$hist <- renderPlot({
  hist(rv$data, main=rv$titulo)
})
output$texto <- renderPrint({
  cat("Cantidad de muestras generadas: ", length(rv$data))
})
}

shinyApp(ui = ui, server = server)
```

```